

Основы функционального программирования

Практическая работа №2 Функции высших порядков

Цель: Освоить применение `map`, `filter`, `foldr/l`, лямбда-выражений и композиции функций..

Теоретическая справка

- `map :: (a -> b) -> [a] -> [b]` - преобразует каждый элемент списка.
- `filter :: (a -> Bool) -> [a] -> [a]` - оставляет элементы по предикату.
- `foldr :: (a -> b -> b) -> b -> [a] -> b` - сворачивает список справа.
- Лямбды: `\x -> x * 2`
- Композиция: `(f . g) x = f (g x)`
- Каррирование: `zipWith (*) [1,2]` возвращает функцию `[Int] -> [Int]`

Пример решения

Задача: Реализовать `wordLengths :: String -> [Int]` - список длин слов в строке.

```
import Data.Char (isSpace)

-- Вспомогательная функция: разбить строку на слова без
использования words

splitWords :: String -> [String]
splitWords = foldr step [[]]
  where
    step c (acc:rest)
      | isSpace c = [] : acc : rest  -- если пробел, то новый
список
      | otherwise = (c:acc) : rest
    step _ [] = [[]]  -- защита от пустого аккумулятора
```

```
-- Основная функция
wordLengths :: String -> [Int]
wordLengths = map length . filter (not . null) . splitWords
```

Тестирование в GHCi

```
> wordLengths "Hello World !"
[5,5,1]
```

Ход работы

1. Создайте файл Pr2_V<N>.hs (где <N> — номер вашего варианта)..
2. Для каждой функции из варианта:
 - a. Используйте только функции высших порядков (map, filter, foldr/l, zipWith)
 - b. Избегайте явной рекурсии
 - c. Применяйте композицию (.) и частичное применение
3. Протестируйте все функции в GHCi.
4. Оформите отчёт (см. ниже).

Требования к отчёту

- Титульный лист (ФИО, группа, работа №1, вариант)
- Условие задачи
- Листинг кода с комментариями
- Примеры вызовов с результатами (скриншоты из GHCi или текст)
- Выводы: какие конструкции использовались, с какими трудностями столкнулись
- Формат PDF

Варианты заданий

Важно!

- Запрещена явная рекурсия (нет самовывоза функции)
- Обязательно использовать как минимум две разные функции высших порядков на задачу
- Разрешено использовать `import Data.Char`, `Data.List` (но не `group`, `sort` и т.п. без согласования)

Вариант 1

Функция	Описание	Примеры
<code>normalize :: [Double] -> [Double]</code>	Нормализует числа к диапазону [0,1]: $(x - \min) / (\max - \min)$. Если все элементы равны - все значения 0.0.	<code>normalize [1,2,3] → [0.0,0.5,1.0]</code> <code>normalize [5,5,5] → [0.0,0.0,0.0]</code>
<code>countLongWords :: String -> Int</code>	Количество слов длиной ≥ 6 символов. Слова - последовательности непробельных символов.	<code>countLongWords "Haskell programming language" → 2</code> <code>countLongWords "hi bye" → 0</code>

Вариант 2

Функция	Описание	Примеры
<code>movingAverage :: Int -> [Double] -> [Double]</code>	Скользящее среднее с окном размера n . Для позиции i берётся среднее элементов $[i-n+1 .. i]$. Первые $n-1$ элементов не входят в результат.	<code>movingAverage 3 [1,2,3,4,5] → [2.0,3.0,4.0]</code>
<code>filterPrime :: [Int] -> [Int]</code>	Фильтрация простых чисел (≥ 2 , делятся только на 1 и себя)	<code>filterPrime [1,2,3,4,5,6,7] → [2,3,5,7]</code> <code>filterPrime [-5,0,1] → []</code>

Вариант 3

Функция	Описание	Примеры
<code>wordFrequencies :: String -> [(String,Int)]</code>	Частота слов в строке (без учёта регистра). Слова приводить к нижнему регистру. Сортировка не требуется.	<code>wordFrequencies "Hello hello world" → [("hello",2),("world",1)]</code> или <code>[("world",1),("hello",2)]</code>
<code>cumulativeSum :: [Int] -> [Int]</code>	Префиксные суммы: $[a_1, a_2, a_3, \dots] \rightarrow [a_1, a_1+a_2, a_1+a_2+a_3, \dots]$	<code>cumulativeSum [1,2,3,4] → [1,3,6,10]</code> <code>cumulativeSum [] → []</code>

Вариант 4

Функция	Описание	Примеры
<code>scaleMatrix :: Double -> [[Double]] -> [[Double]]</code>	Умножает каждый элемент матрицы (списка списков) на скаляр.	<code>scaleMatrix 2</code> <code>[[1,2],[3,4]] →</code> <code>[[2.0,4.0],[6.0,8.0]]</code>
<code>dropWhileEnd :: (a -> Bool) -> [a] -> [a]</code>	Удаляет элементы с конца списка, пока выполняется предикат (аналог <code>dropWhile</code> для конца).	<code>dropWhileEnd (== ' ')</code> <code>"hello " → "hello"</code> <code>dropWhileEnd even</code> <code>[2,4,6,3,4,6] →</code> <code>[2,4,6,3]</code>

Вариант 5

Функция	Описание	Примеры
<code>histogram :: [Int] -> [(Int,Int)]</code>	Строит гистограмму: для каждого уникального значения — количество его вхождений. Порядок не важен.	<code>histogram [1,2,1,3,2,1]</code> <code>→ [(1,3),(2,2),(3,1)]</code>
<code>scanr1Safe :: (a -> a -> a) -> [a] -> [a]</code>	Аналог <code>scanr1</code> , но для пустого списка возвращает <code>[]</code> (а не <code>error</code>).	<code>scanr1Safe (+) [1,2,3]</code> <code>→ [6,5,3]</code> <code>scanr1Safe (+) [] → []</code>

Вариант 6

Функция	Описание	Примеры
<code>replaceIf :: (a -> Bool) -> a -> [a] -> [a]</code>	Заменяет все элементы, удовлетворяющие предикату, на значение по умолчанию.	<code>replaceIf even 0</code> <code>[1,2,3,4] → [1,0,3,0]</code> <code>replaceIf (== 'a') 'x'</code> <code>"banana" → "bxnxbn"</code>
<code>dotProduct :: [Double] -> [Double] -> Double</code>	Скалярное произведение векторов. Если длины разные — использовать минимальную длину (как <code>zip</code>).	<code>dotProduct [1,2,3]</code> <code>[4,5,6] → 32.0 (1·4 + 2·5 + 3·6)</code> <code>dotProduct [1,2] [3,4,5]</code> <code>→ 11.0</code>

Вариант 7

Функция	Описание	Примеры
<code>groupLengths :: Eq a => [a] -> [Int]</code>	Длины групп подряд идущих одинаковых элементов (аналог <code>map length . group</code>).	<code>groupLengths "aaabbccccc" → [3,2,4]</code> <code>groupLengths [1,1,2,2,2,3] → [2,3,1]</code>
<code>zipWith3 :: (a->b->c->d) -> [a] -> [b] -> [c] -> [d]</code>	Обобщение <code>zipWith</code> для трёх списков. Обрезает по минимальной длине.	<code>zipWith3 (\a b c -> a+b+c) [1,2] [3,4] [5,6] → [9,12]</code>

Вариант 8

Функция	Описание	Примеры
<code>partition3 :: (a -> Bool) -> (a -> Bool) -> [a] -> ([a],[a],[a])</code>	Разбивает список на три части: 1. Элементы, удовлетворяющие первому предикату 2. Элементы, удовлетворяющие второму, но не первому 3. Остальные элементы	<code>partition3 even (>5) [1,2,6,7,8] → ([2,6,8],[7],[1])</code>
<code>geometricMean :: [Double] -> Double</code>	Среднее геометрическое: $\sqrt[n]{x_1 \cdot x_2 \cdot \dots \cdot x_n}$. Для пустого списка — <code>0.0</code> .	<code>geometricMean [1,2,8] → 2.5198... ($\sqrt[3]{16}$)</code> <code>geometricMean [4,4] → 4.0</code>

Вариант 9

Функция	Описание	Примеры
<code>suffixes :: [a] -> [[a]]</code>	Все суффиксы списка в порядке убывания длины (от исходного до пустого).	<code>suffixes [1,2,3] → [[1,2,3],[2,3],[3],[]]</code> <code>suffixes "abc" → ["abc","bc","c",""]</code>
<code>countInversions :: Ord a => [a] -> Int</code>	Количество инверсий: пар (i,j) , где $i < j$ и $a_i > a_j$.	<code>countInversions [2,4,1,3,5] → 3 (2>1, 4>1, 4>3)</code> <code>countInversions [1,2,3] → 0</code>

Вариант 10

Функция	Описание	Примеры
<code>slidingWindow :: Int -> [a] -> [[a]]</code>	Скользящее окно размера <code>n</code> : список подсписков длиной <code>n</code> , сдвинутых на 1 позицию.	<code>slidingWindow 3</code> <code>[1,2,3,4,5] →</code> <code>[[1,2,3],[2,3,4],[3,4,5]]</code>
<code>correlation :: [Double] -> [Double] -> Double</code>	Коэффициент корреляции Пирсона. Для простоты можно реализовать упрощённую версию или вернуть <code>0.0</code> для списков разной длины/пустых.	<code>correlation [1,2,3]</code> <code>[1,2,3] → 1.0</code> <code>correlation [1,2,3]</code> <code>[3,2,1] → -1.0</code>

Вариант 11

Функция	Описание	Примеры
<code>removeOutliers :: Double -> [Double] -> [Double]</code>	Удаляет выбросы по правилу межквартильного размаха (IQR): $Q1 - k \cdot IQR$ и $Q3 + k \cdot IQR$, где $IQR = Q3 - Q1$, k — коэффициент. Для упрощения можно использовать фиксированные границы или упрощённую логику.	<code>removeOutliers 1.5</code> <code>[1,2,3,4,100] →</code> <code>[1,2,3,4]</code>
<code>isPermutation :: Eq a => [a] -> [a] -> Bool</code>	Проверяет, является ли второй список перестановкой первого (одинаковые элементы с учётом кратности).	<code>isPermutation [1,2,3]</code> <code>[3,1,2] → True</code> <code>isPermutation "abc"</code> <code>"aab" → False</code>

Вариант 12

Функция	Описание	Примеры
<code>polynomialValue :: [Double] -> Double -> Double</code>	Вычисляет значение полинома. Коэффициенты от старшего к младшему: $[a_n, \dots, a_1, a_0] \rightarrow a_n \cdot x^n + \dots + a_1 \cdot x + a_0$.	<code>polynomialValue [1,0,-2,3] 2</code> → <code>1·8 + 0·4 - 2·2 + 3 = 7</code>
<code>transposeWith :: (a -> b -> c) -> [[a]] -> [[b]] -> [[c]]</code>	Транспонирование с применением функции: элемент <code>[i][j]</code> результата = <code>f (mat1[j][i]) (mat2[j][i])</code> . Для упрощения можно реализовать как <code>zipWith (zipWith f)</code> после транспонирования.	<code>transposeWith (+) [[1,2],[3,4]] [[10,20],[30,40]]</code> → <code>[[11,32],[23,44]]</code>

Вариант 13

Функция	Описание	Примеры
<code>rleDecode :: [(a,Int)] -> [a]</code>	Декодирование RLE: распаковка списка пар (значение, количество) в плоский список.	<code>rleDecode [('a',3),('b',2)]</code> → <code>"aaabb"</code> <code>rleDecode [(1,2),(2,3)]</code> → <code>[1,1,2,2,2]</code>
<code>prefixSums2D :: [[Int]] -> [[Int]]</code>	Двумерные префиксные суммы: элемент <code>[i][j]</code> = сумма всех элементов <code>[0..i][0..j]</code> исходной матрицы.	<code>prefixSums2D [[1,2],[3,4]]</code> → <code>[[1,3],[4,10]]</code>

Вариант 14

Функция	Описание	Примеры
<code>sieveOfEratosthenes :: Int -> [Int]</code>	Генерация простых чисел до <code>n</code> включительно методом решета Эратосфена.	<code>sieveOfEratosthenes 10</code> → <code>[2,3,5,7]</code> <code>sieveOfEratosthenes 1</code> → <code>[]</code>
<code>cartesianProduct :: [a] -> [b] -> [(a,b)]</code>	Декартово произведение двух списков: все пары (x,y) , где $x \in xs, y \in ys$.	<code>cartesianProduct [1,2] ['a','b']</code> → <code>[(1,'a'),(1,'b'),(2,'a'),(2,'b')]</code>

Вариант 15

Функция	Описание	Примеры
<code>levenshtein :: String -> String -> Int</code>	Расстояние Левенштейна — минимальное количество операций вставки, удаления, замены для преобразования одной строки в другую. Для упрощения можно реализовать рекурсивную версию с мемоизацией через <code>Data.Array</code> или ограничить длину строк.	<code>levenshtein "kitten"</code> <code>"sitting" → 3</code> <code>levenshtein "abc" "abc"</code> <code>→ 0</code>
<code>matrixMultiply :: [[Double]] -> [[Double]] -> [[Double]]</code>	Умножение матриц. Предполагается корректность размеров (число столбцов первой = числу строк второй).	<code>matrixMultiply</code> <code>[[1,2],[3,4]]</code> <code>[[5,6],[7,8]] →</code> <code>[[19,22],[43,50]]</code>